

INFO 410/510 Bayesian Modelling and Inference

Lecture 12 – Monte Carlo Methods (MCMC algorithm)

Dr. Kunal Arekar
College of Information
University of Arizona, Tucson

October 27, 2025

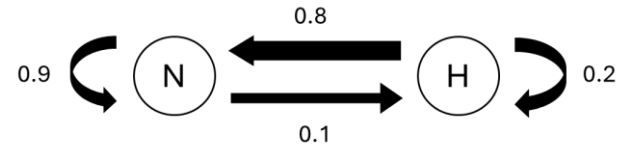


Markov Chain Monte Carlo

- MCMC methods are a family of algorithms that uses Markov Chains to perform Monte Carlo estimate
- Class of algorithms used to draw samples from a probability distribution
- Important and popular concepts in Bayesian Statistics, especially while doing inference

Markov chains

Quick review



Nestle = 0.6

Hershey = 0.4

		t + 1	
		N	H
t	N	0.9	0.1
	H	0.8	0.2

Transition matrix

At time t + 1, what market share will Nestle and Hershey have?

$$\begin{aligned}
 N(t + 1) &= (0.6 * 0.9) + (0.4 * 0.8) \\
 &= 0.54 + 0.32 = 0.86
 \end{aligned}$$

$$\begin{aligned}
 H(t + 1) &= (0.4 * 0.2) + (0.6 * 0.1) \\
 &= 0.08 + 0.06 = 0.14
 \end{aligned}$$

	N	H
t	0.6	0.4
t + 1		
t + 2		

Markov chain is a process that consists of a finite number of states and some known probabilities p_{ij} , where p_{ij} is the probability of moving from state i to state j . Transition probability (transition matrix)

Markov Property and Memorylessness

The key feature of Markov Chains is that the future state depends only on the current state, not on the sequence of events that preceded it. This property is called the Markov Property.

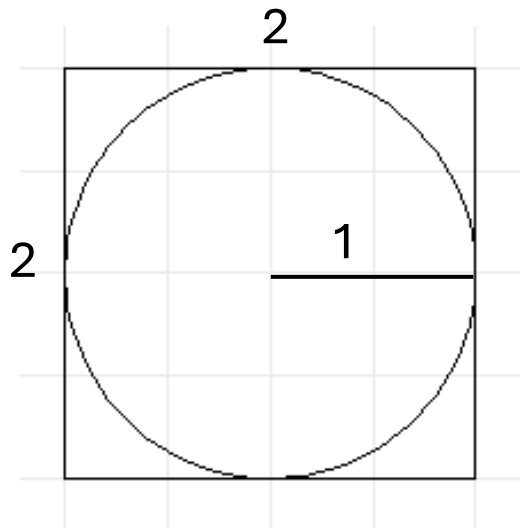
$$P(X_{t+1} = s \mid X_t = s_t, \cancel{X_{t-1} = s_{t-1}}, \dots, \cancel{X_0 = s_0}) = P(X_{t+1} = s \mid X_t = s_t)$$

distribution
of X_{t+1}

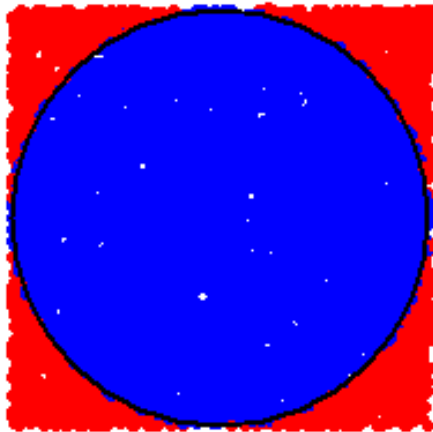
depends
on X_t

whatever happened before
time t doesn't matter.

Monte Carlo methods



Prob of a random point inside the circle = $\pi/4$ = **0.7853982**



```
N <- 1000000
```

```
circ <- 0
```

```
for (i in 1:N) {
```

```
  x <- runif(1, -1, 1)
```

```
  y <- runif(1, -1, 1)
```

```
  if (x^2 + y^2 <= 1) {
```

```
    circ <- circ + 1
```

```
  }
```

```
}
```

```
test <- circ/N = 0.785313
```

~ 3 secs

N (10) = 0.9

N (100) = 0.81

N (10000) = 0.7903

I play a game with $P(\text{win}) = p$, for each round

Game ends if I lose 2 times in a row

e.g., $p, (1-p), (1-p)$

What is the expected number of rounds I play?

Let $e = E(R)$

$$e = (1+e) * p + \\ 2 * (1-p)(1-p) + \\ (2+e) * (1-p) * p$$

$$e = (2-p)/(1-p)^2$$

```
rounds <- vector()
for (i in 1:1000000) {
  r <- 0
  nloss <- 0
  while (nloss != 2) {
    r <- r + 1
    if (runif(1) < p) {
      nloss <- 0
    }
    else {
      nloss <- nloss + 1
    }
  }
  rounds <- c(rounds, r)
}
mean_rounds <- mean(rounds)
mean_rounds
```

If $p = 0.5$

true $e = 6$

estimated = 5.996614

time : ~ 11 sec

Monte Carlo methods

Problems where an exact solution is difficult or impossible to compute directly

Such as - high-dimensional integrals, complex probabilistic models, or large-scale simulations

For e.g.

- Simulating stock prices - how a portfolio of stocks will behave under different market conditions, randomly varying interest rates, stock prices, and other factors (finance)
- simulate complex systems - how particles move and interact (physics)

Use Monte Carlo methods which solves problems through random sampling

useful when dealing with complex systems where a deterministic solution (i.e., one where every step is fully calculated) is either impossible or computationally expensive

A bit of history

Late 1940s, early 1950s:
Manhattan project

Difficulty: complex simulations of
neutrons in nuclear bombs

Supposedly, Ulam was playing solitaire and
realized that they could estimate fixed
quantities by random simulation
(*What are the chances that a randomly shuffled
games of solitaire could be won?*)

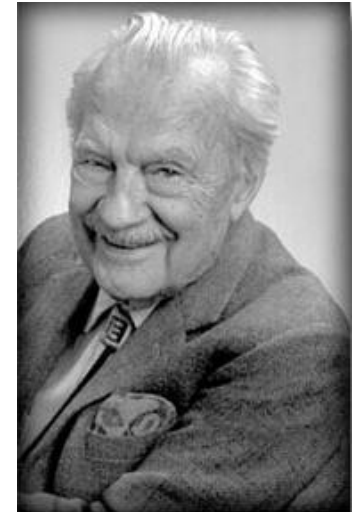
Needed a code name: Metropolis suggested
Monte Carlo



Monte Carlo Casino



Stanisław Ulam



Nicholas Metropolis

Monte Carlo integration

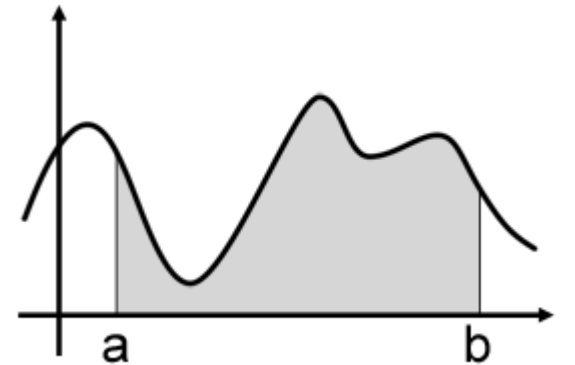
solving integrals having numerous values to integrate upon

$$F = \int_a^b f(x)dx$$

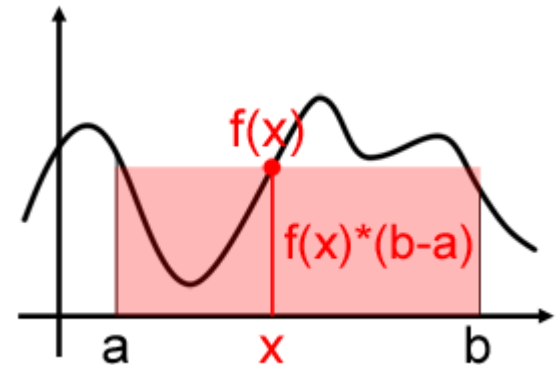
Integrating a function is graphically equivalent to calculating the area under the curve of that function

The idea is to estimate the integral of a function, over a defined interval, only knowing the function expression

Monte Carlo integration is a technique for numerical integration using random sampling and law of large numbers



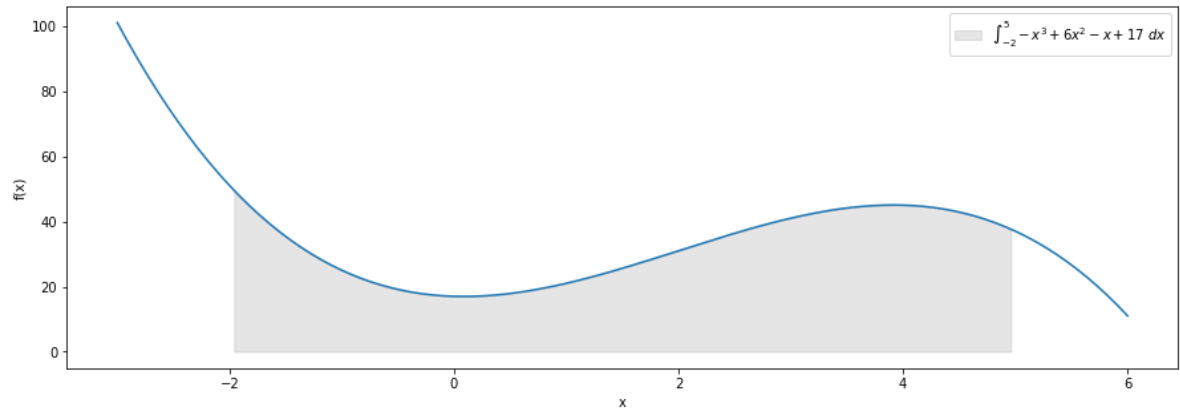
© www.scratchapixel.com



the curve can be evaluated at x and the result can be multiplied by $(b - a)$. This defines a rectangle which can be seen as a very crude approximation of the integral.

$$f(x) = -x^3 + 6x^2 - x + 17$$

Let's say $a = -2$ and $b = 5$

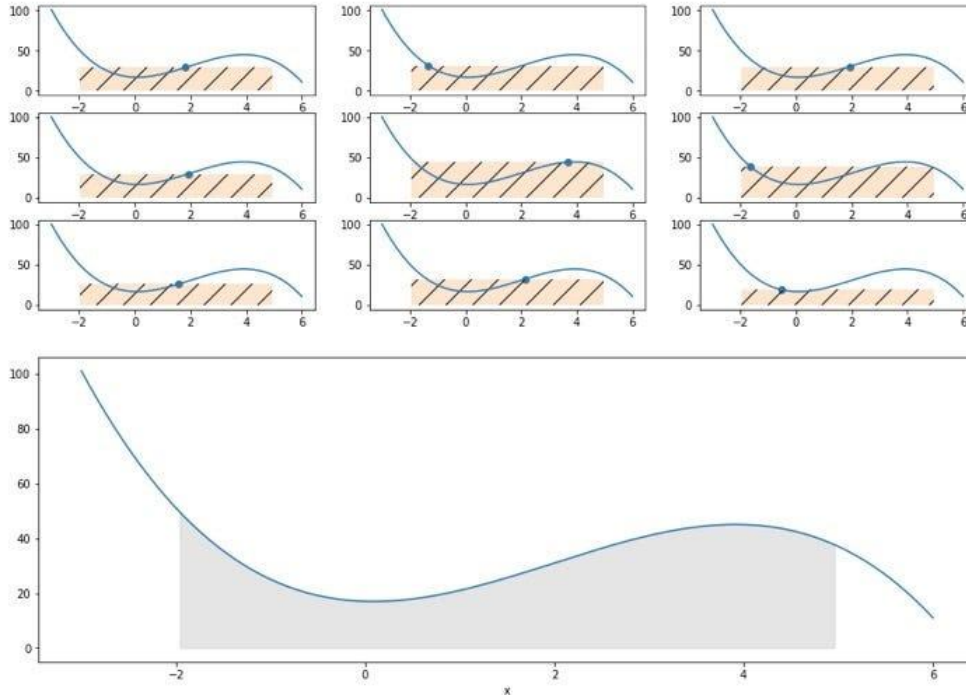


If we take a random point x_i between a and b , we can multiply $f(x_i)$ by $(b-a)$ to get the area of a rectangle of width $(b-a)$ and height $f(x_i)$

$$f(x_i) \cdot (b-a)$$

The idea behind Monte Carlo integration is to approximate the integral value (gray area) by the averaged area of rectangles computed for random picked x_i

The idea behind Monte Carlo integration is to approximate the integral value (gray area) by the averaged area of rectangles computed for randomly picked x_i



adding up the area of the rectangles and averaging the sum, the resulting number gets closer and closer to the actual result of the integral

$$\langle F^N \rangle = (b - a) \frac{1}{N-1} \sum_{i=0}^N f(X_i)$$

Monte Carlo estimator

$$Pr \left\{ \lim_{N \rightarrow \infty} \langle F^N \rangle = F \right\} = 1$$

as N approaches infinity, the Monte Carlo estimator converges in probability to F , the true value of the integral

variance of the estimator: σ^2 / N

It means we need 4 times more samples to reduce the error of the estimator by 2

Crude Monte Carlo Method

Most basic and foundational form of Monte Carlo methods

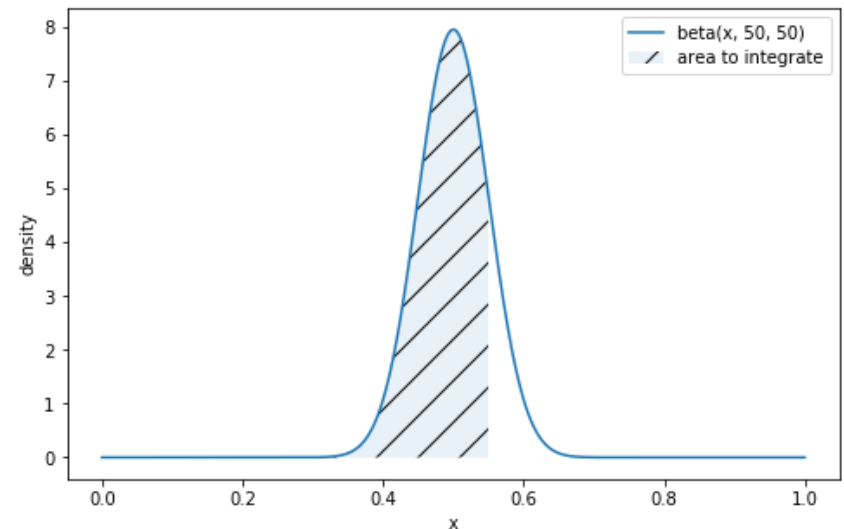
- Random draws x_i are made over X following a uniform law
- We compute the sum of $f(x_i)$, multiply it by $(b-a)$ and divide by the number of samples N .

$$\langle F^N \rangle = (b-a) \frac{1}{N-1} \sum_{i=0}^N f(X_i)$$

$$f(x, 50, 50) = \frac{1}{B(50, 50)} x^{49} (1-x)^{49} \mathbb{I}_{[0,1]}(x)$$

If X is a random variable that follows this beta law, this integral corresponds to $P(X \leq 0.55)$

cumulative density function (c.d.f.) of the $\text{beta}(x, 50, 50)$ density law: gives **0.8413**



we want to integrate the beta distribution function $\text{beta}(x, 50, 50)$ over the interval $[0 ; 0.55]$

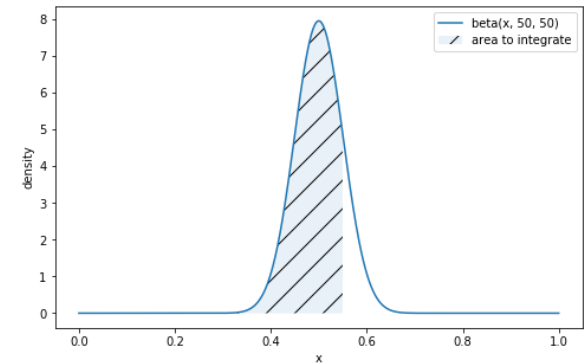
Monte Carlo estimator

$N = 10\,000$ random draws of x_i

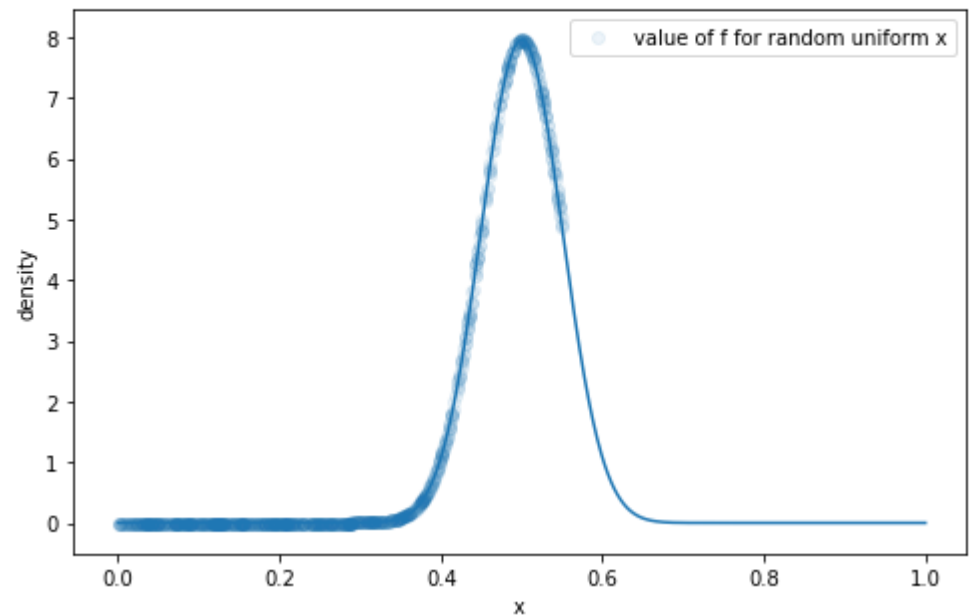
compute the integral estimation and the associated error

$$I = \frac{b-a}{N} \sum_{i=0}^N f(x_i)$$

$$Error = \sqrt{\frac{\frac{b-a}{N} \sum_{i=0}^N f^2(x_i) - (b-a)\mu^2}{N}} = \frac{\sigma}{\sqrt{N}}$$



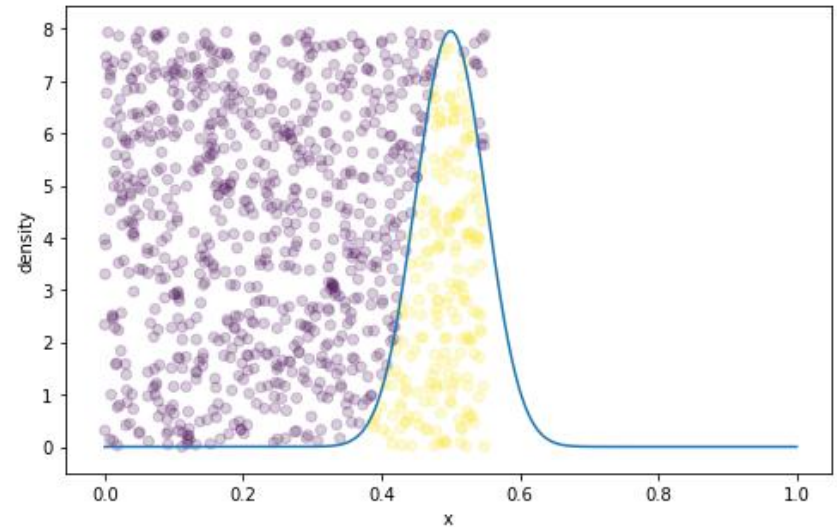
```
Real value to find: 0.8413478010629016  
Integral value: 0.8577474542754701  
Calculation error: 0.019901983962917785
```



Geometric estimation of the area under the curve

$$\int_0^{0.55} f(x, 50, 50) dx = \frac{nb_yellow_points}{N_{mc}} \cdot height \cdot width$$

Integral value: 0.8264543123827801



Another way to compute the integral is to make a geometric approximation of the integral

the ratio of points under the curve over the total points drawn in the rectangle would converge to the area under the curve with N , the number of draws

Two key algorithms

Importance sampling

Sometimes, sampling directly from a target distribution (e.g., a posterior distribution) is difficult

Importance sampling allows us to sample from an easier distribution and re-weight the samples to approximate the target distribution

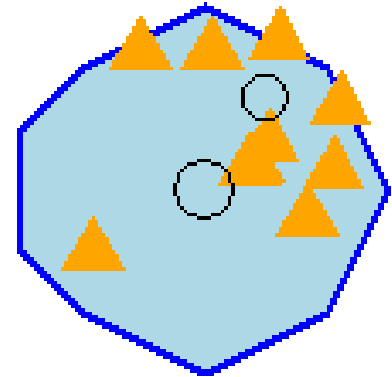
Rejection sampling

Propose samples from an easier distribution

Accept or reject each sample based on how well it fits the target distribution

Importance Sampling

- Let's say you're trying to estimate how many fish there are in a huge pond
- The fish are spread unevenly – gathered at one place (food source), other areas have few fish



Random Sampling –

Randomly toss net

Throw the net >>>many times to eventually hit the fish-heavy spots

Inefficient approach

Importance Sampling (Efficient Approach) –

Throw the net more often in areas where you know there are more fish

Less important areas, sample less frequently

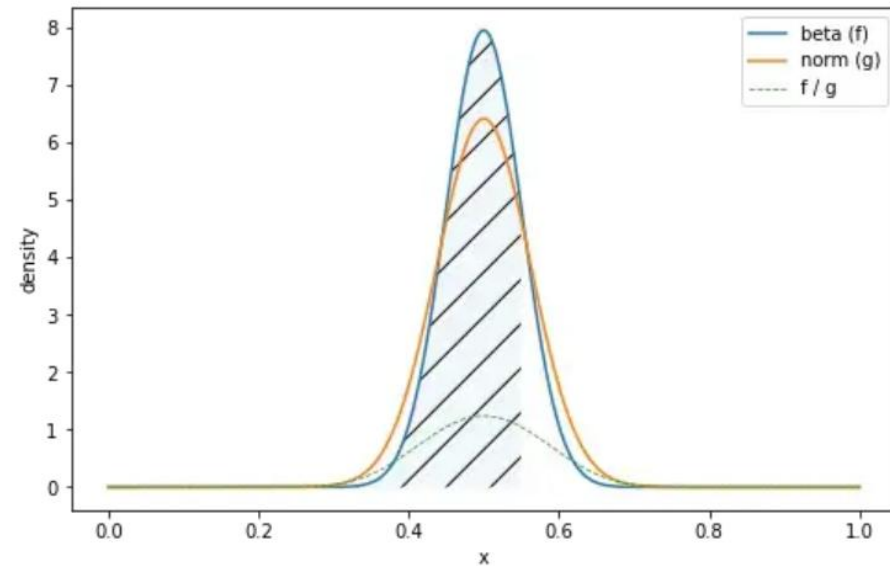
Adjust your results to account for the fact that you biased your sampling toward areas with more fish

Importance Sampling

Mathematical expression

$$I \approx \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{q(x_i)}$$

- x_i are samples from the easy distribution $q(x)$.
- $f(x)$ is the original function you want to integrate
- $q(x)$ is the easier distribution you're sampling from.
- The ratio $f(x)/q(x)$ is the adjustment, or "weight," applied to each sample.



Method	N	Integral	Error
Analytics	/	0.8413	/
Crude	10000	0.8577	0.0199
Importance sampling	10000	0.8421	0.0048

Where Does the Easier Distribution $q(x)$ Come From?

Where Does the Easier Distribution $q(x)$ Come From?

It doesn't just appear magically, it's chosen strategically

The choice depends on the specific problem at hand

Some common guidelines to choose $q(x)$:

Match the Shape of $f(x)$ in “Important” regions

Simpler Functional Forms – $q(x)$ should have a simpler mathematical form that allows easy sampling

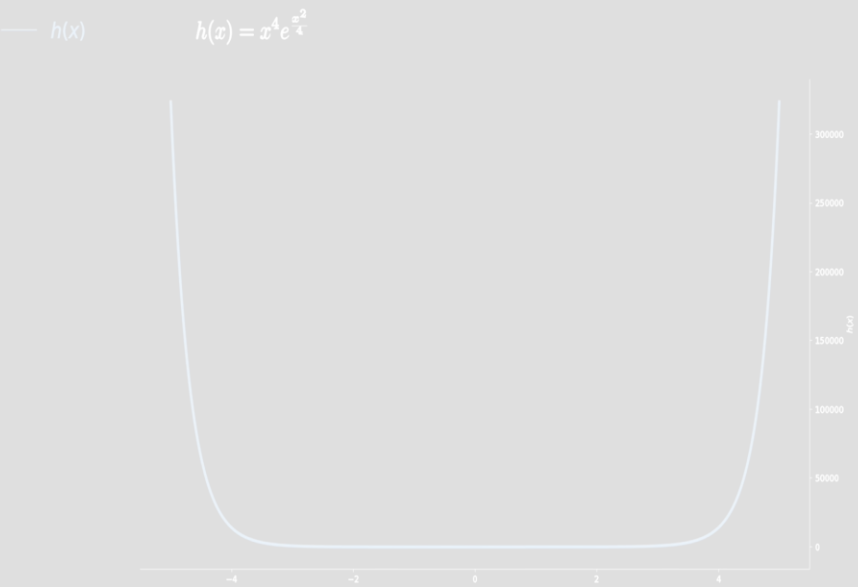
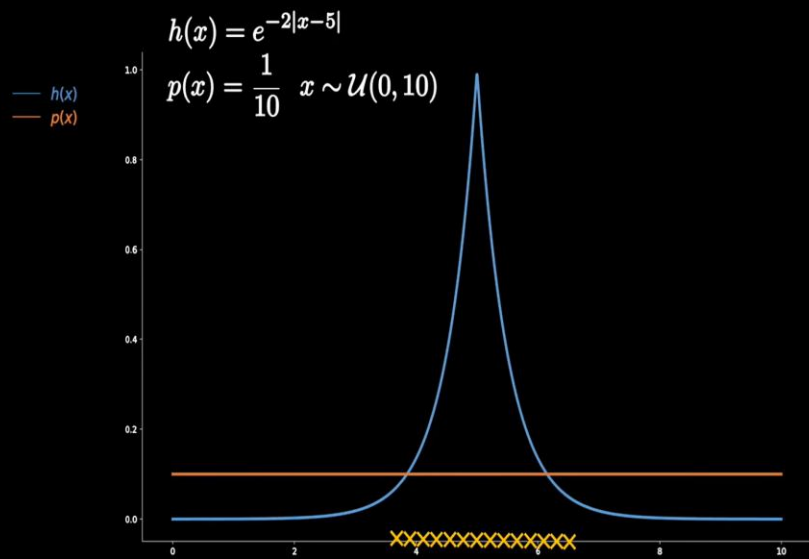
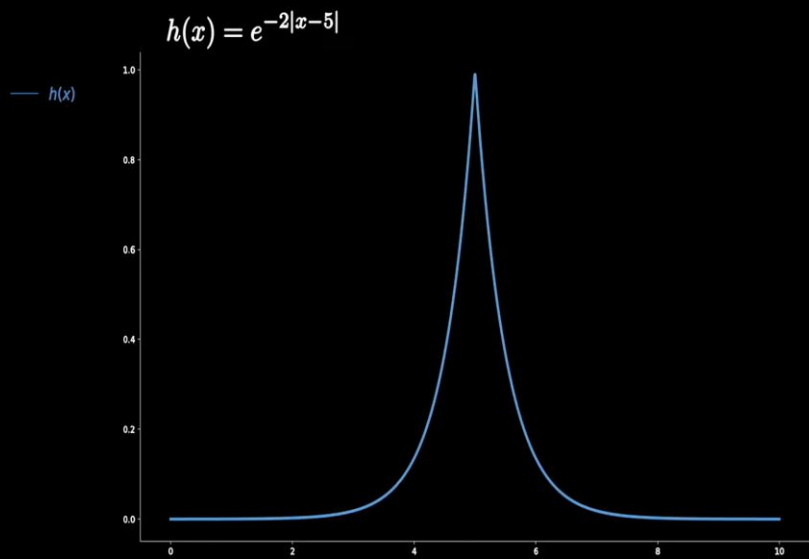
Tail Behavior – if $f(x)$ has long tail, you would want $q(x)$ to have long tail

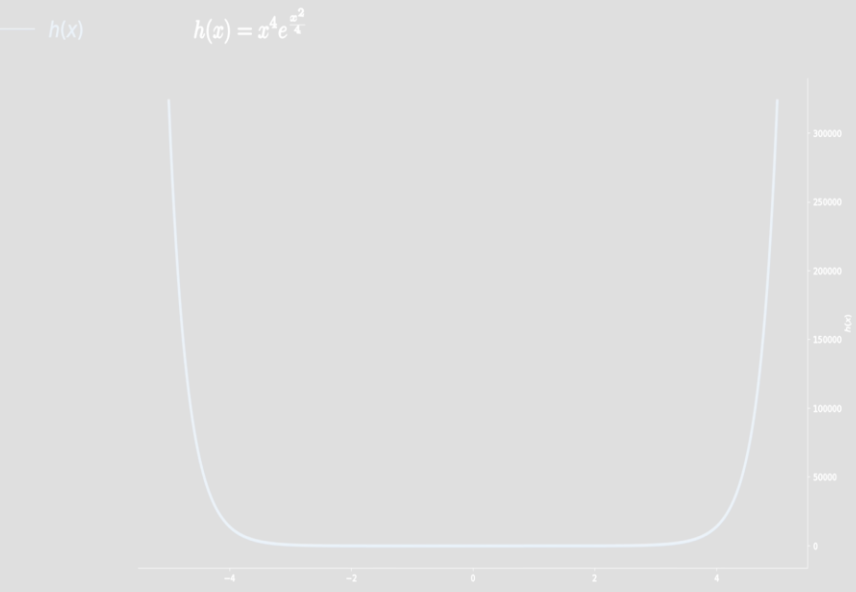
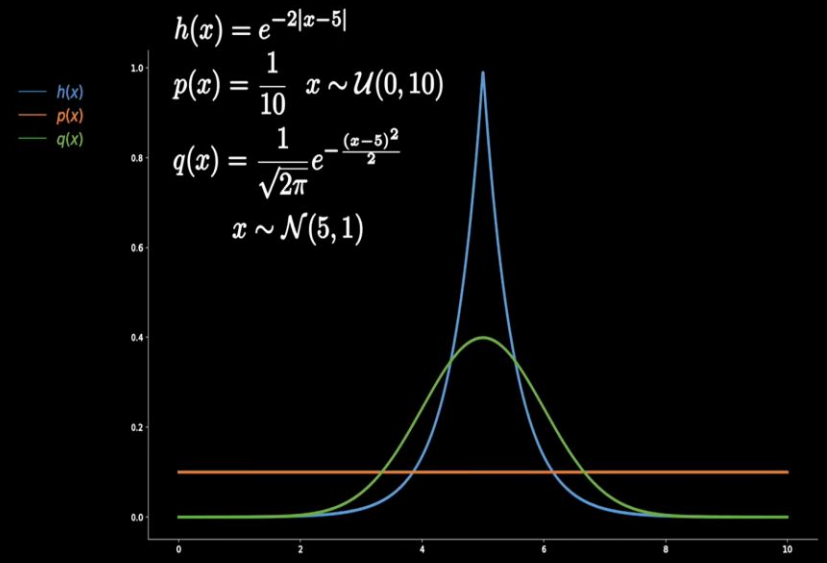
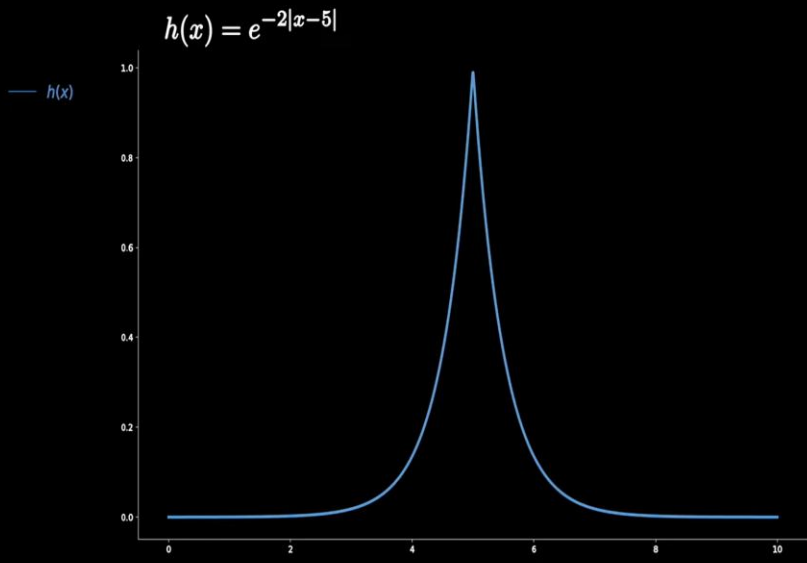
pick one that minimizes the variance of the estimate

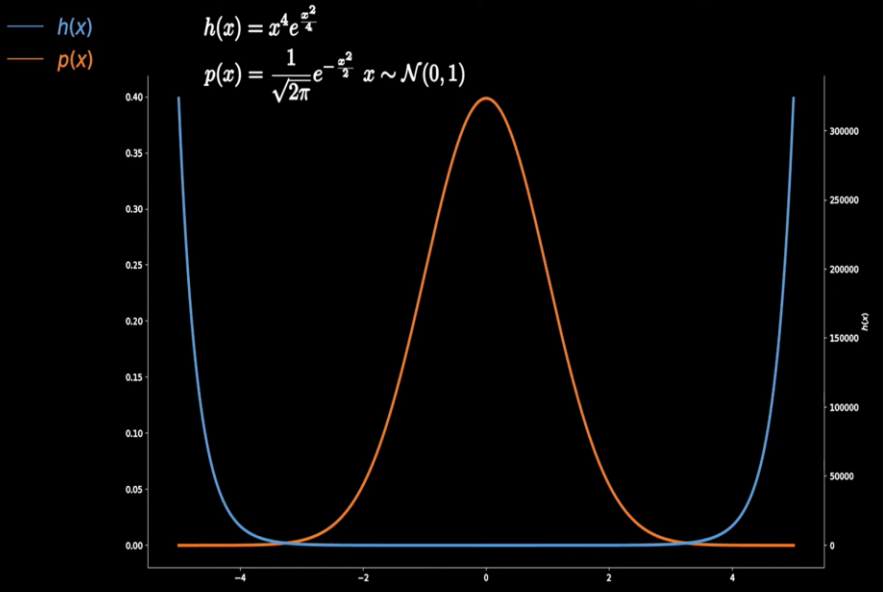
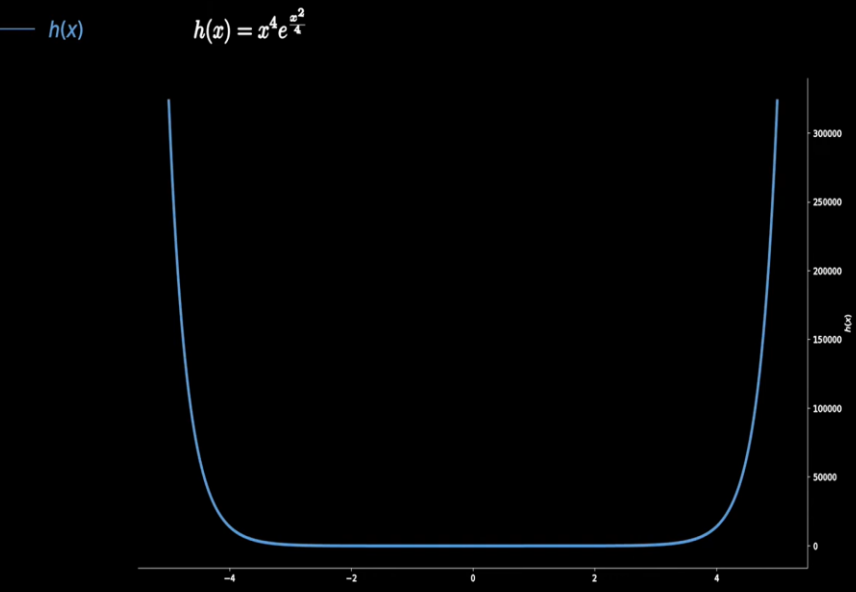
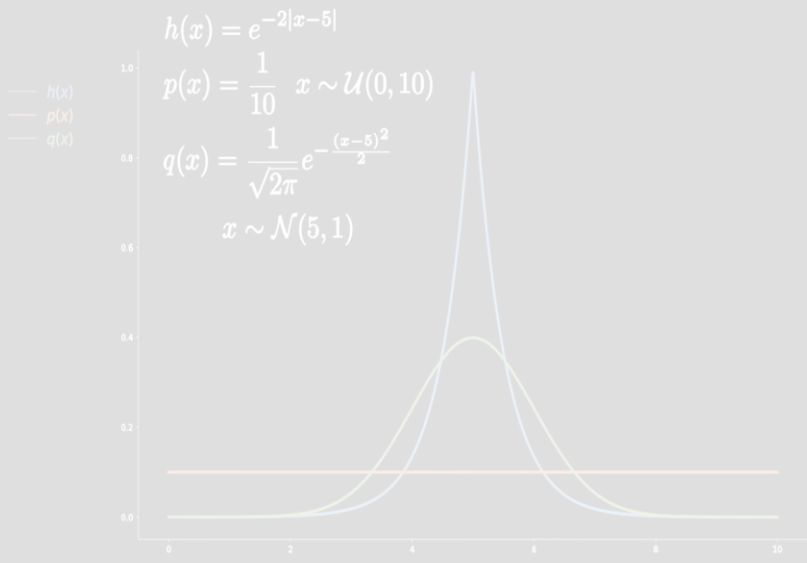
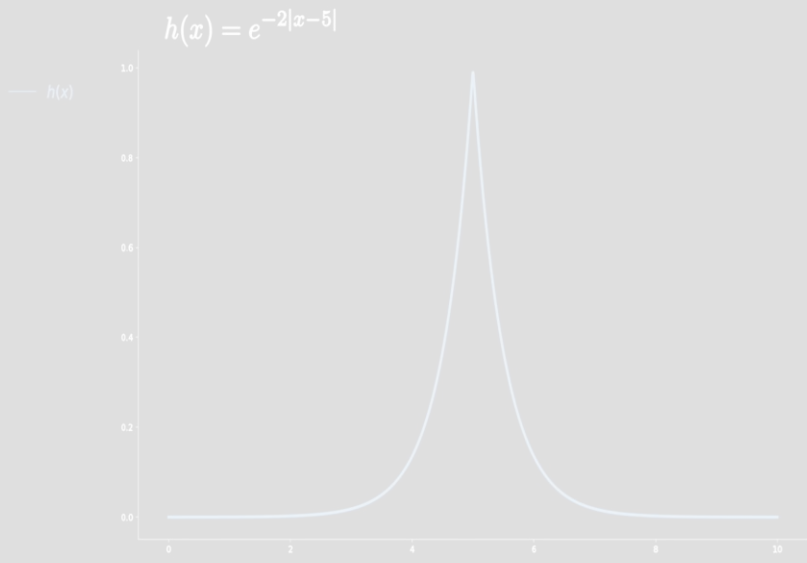
Example -

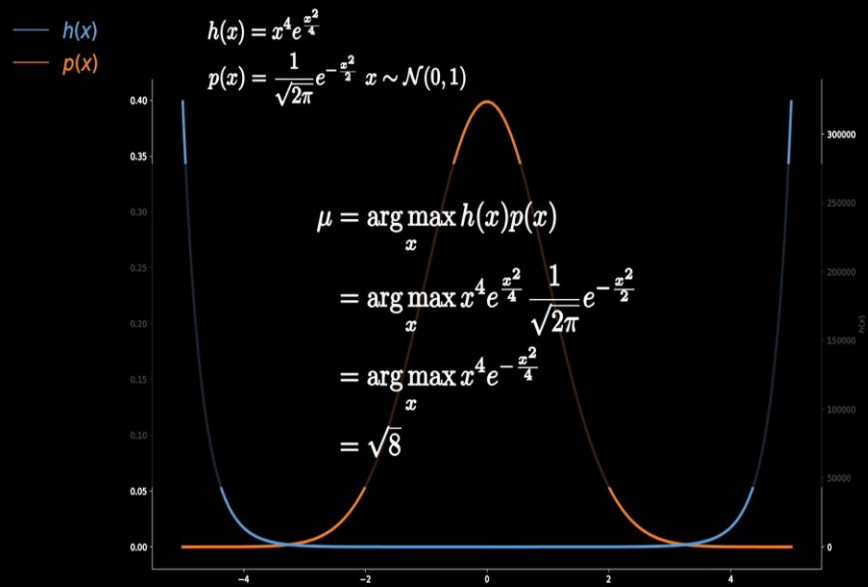
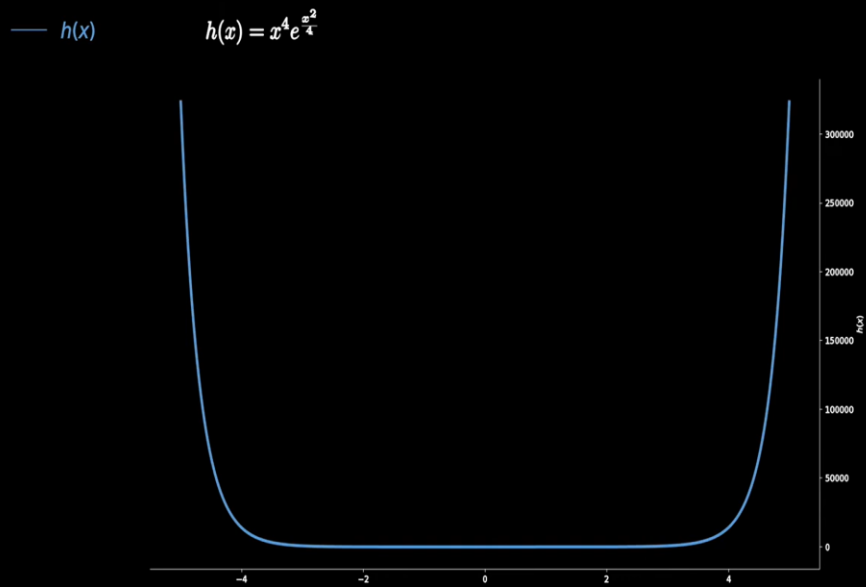
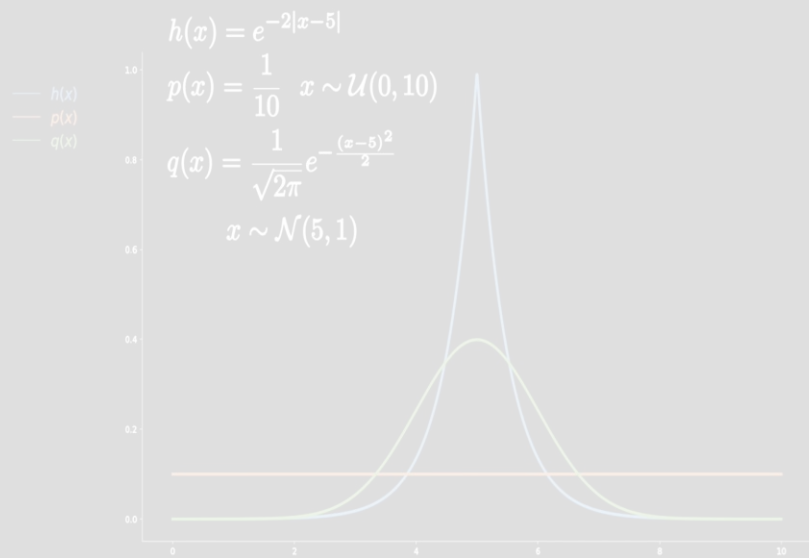
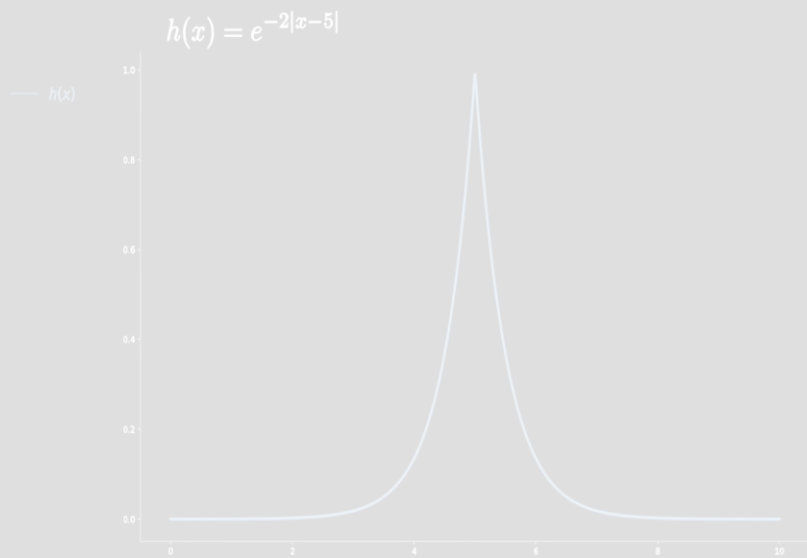
Suppose you're estimating the probability that a stock price falls below a very low threshold (a rare event).

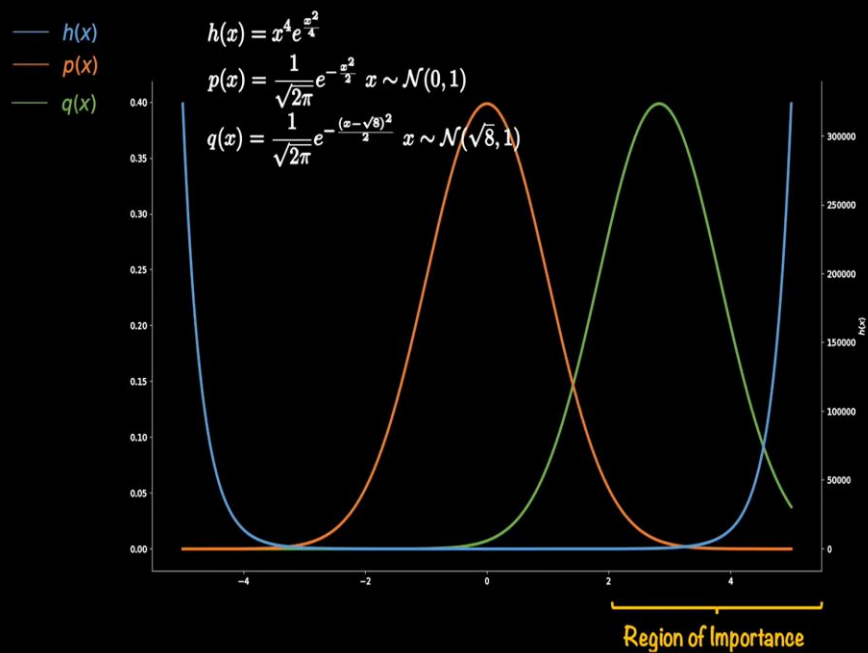
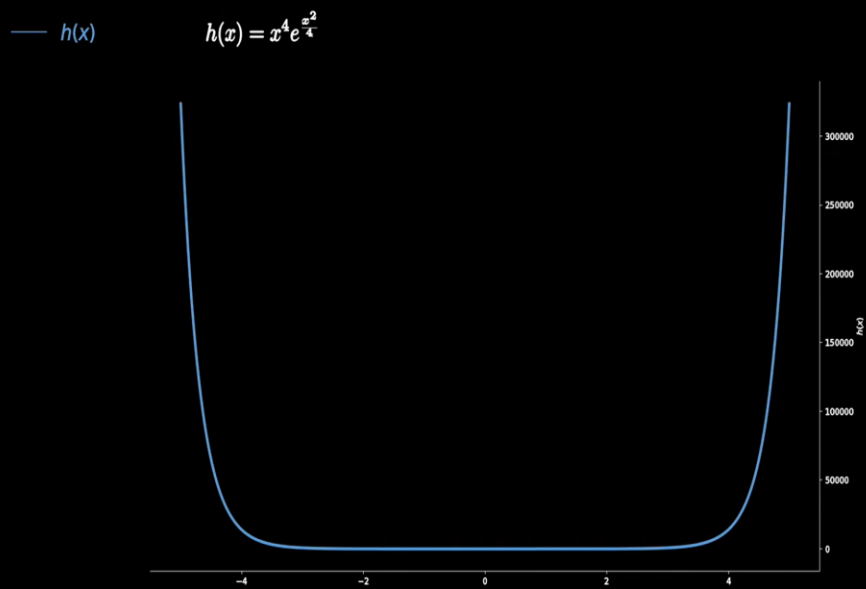
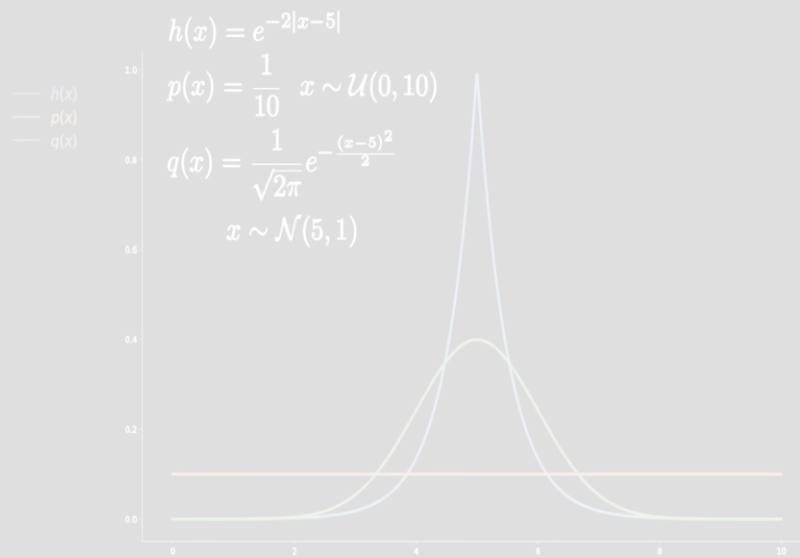
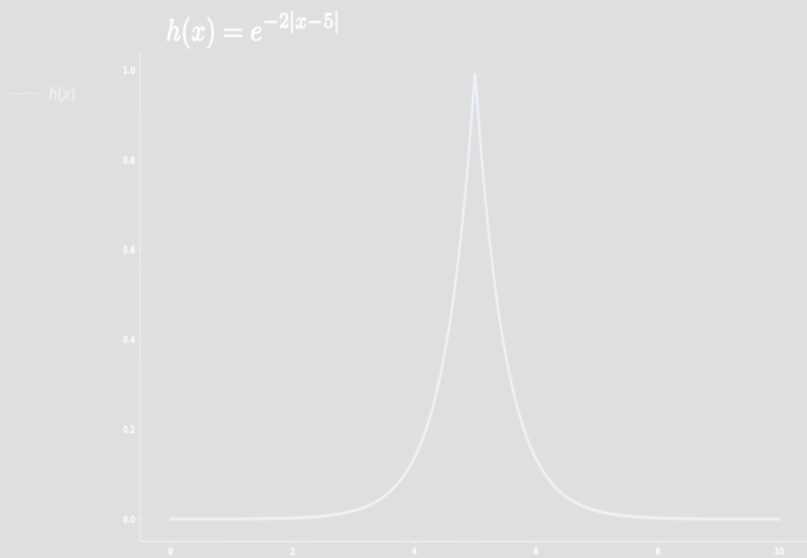
$f(x)$ could be log normal distribution











Two key algorithms

Importance sampling

Sometimes, sampling directly from a target distribution (e.g., a posterior distribution) is difficult

Importance sampling allows us to sample from an easier distribution and re-weight the samples to approximate the target distribution

Rejection sampling



Propose samples from an easier distribution

Accept or reject each sample based on how well it fits the target distribution

Rejection sampling

- **The Target Distribution $f(x)$:** This is the distribution you're interested in, but it's hard to sample from directly (like trying to hit the prize area on the dartboard).
- **The Proposal Distribution $q(x)$:** This is an easier distribution that you can sample from easily.
- **Rejecting and Accepting:**
Once you've generated a random point, you check how "good" the point is based on the target distribution. If it's good (i.e., it falls into the right region), you accept it. If it's not, you reject it.

Steps in Rejection Sampling

complicated distribution $f(x)$

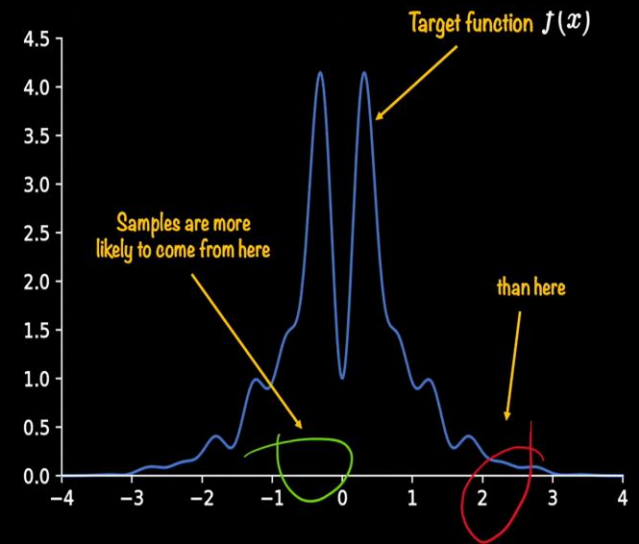
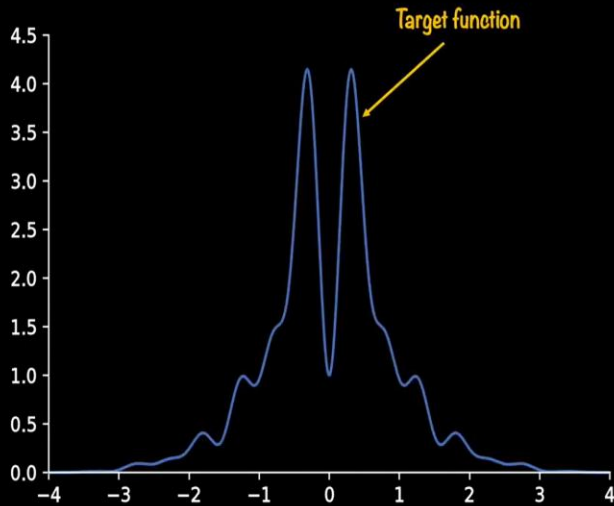
Easy (proposal) distribution $q(x)$

- Sample a point x' from the proposal distribution $q(x)$
- Generate a random number m uniformly from the interval $[0, M]$, where M is a constant chosen so that $Mq(x) \geq f(x)$, for all x
- Accept x' if $m \leq f(x')$
This means the point x' is close enough to being from the target distribution. If the point is rejected, you throw it away and sample again.

$$\frac{P(x')}{Mq(x')}$$

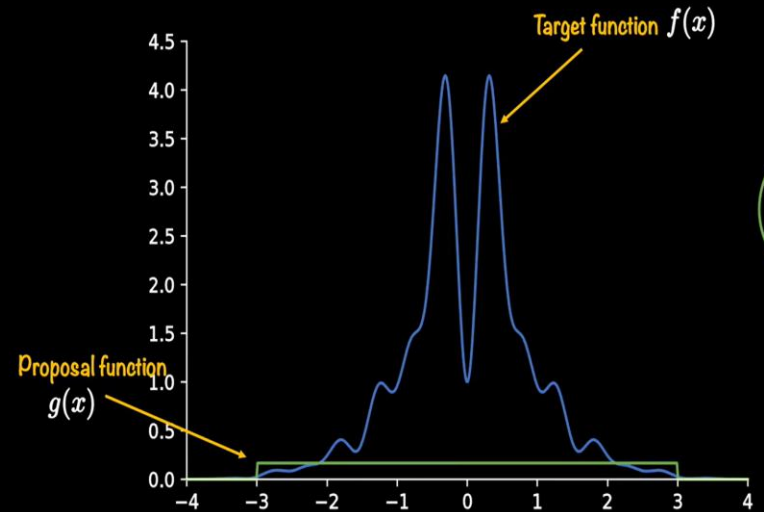
* M is a scaling factor that ensures that $Mq(x)$ "covers" $f(x)$ everywhere.

$$f(x) = e^{-x^2/2} \cdot (\sin^2(6+x) + 3 \cdot \cos^2(x) \cdot \sin^2(4x) + 1)$$



 **Sample from another distribution for the given range**
but ... keep sample only if it satisfies the acceptance criteria

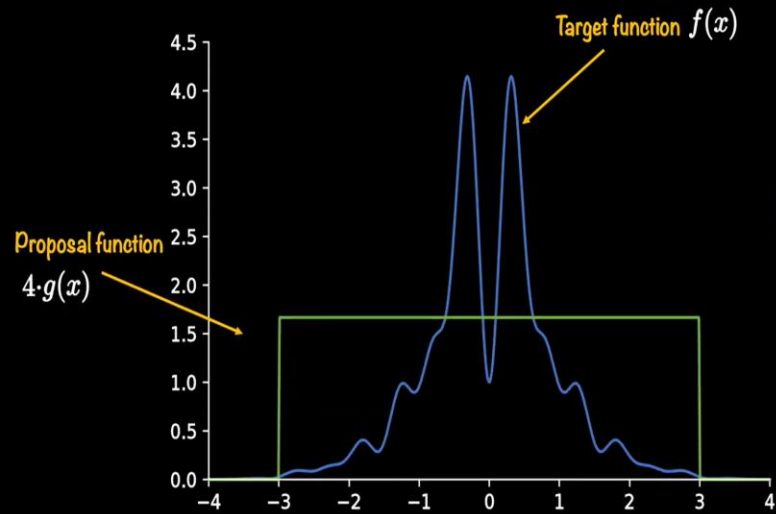
$$f(x) = e^{-x^2/2} \cdot (\sin^2(6+x) + 3 \cdot \cos^2(x) \cdot \sin^2(4x) + 1)$$



$$x \sim \mathcal{U}(-3, 3)$$

$$g(x) = \frac{1}{b-a}$$

$$f(x) = e^{-x^2/2} \cdot (\sin^2(6+x) + 3 \cdot \cos^2(x) \cdot \sin^2(4x) + 1)$$

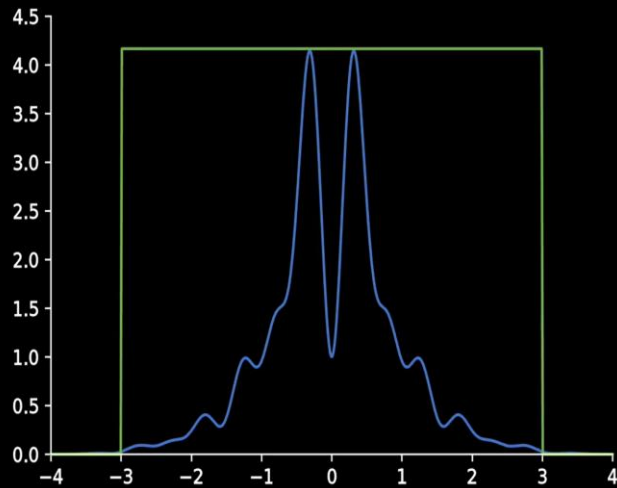


$$x \sim \mathcal{U}(-3, 3)$$

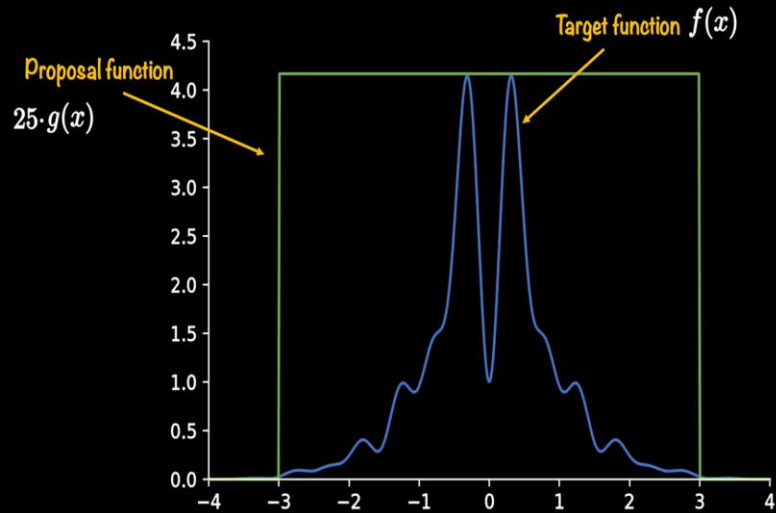
$$g(x) = \frac{1}{b-a}$$

$$\text{— } f(x)$$

$$\text{— } g(x)$$

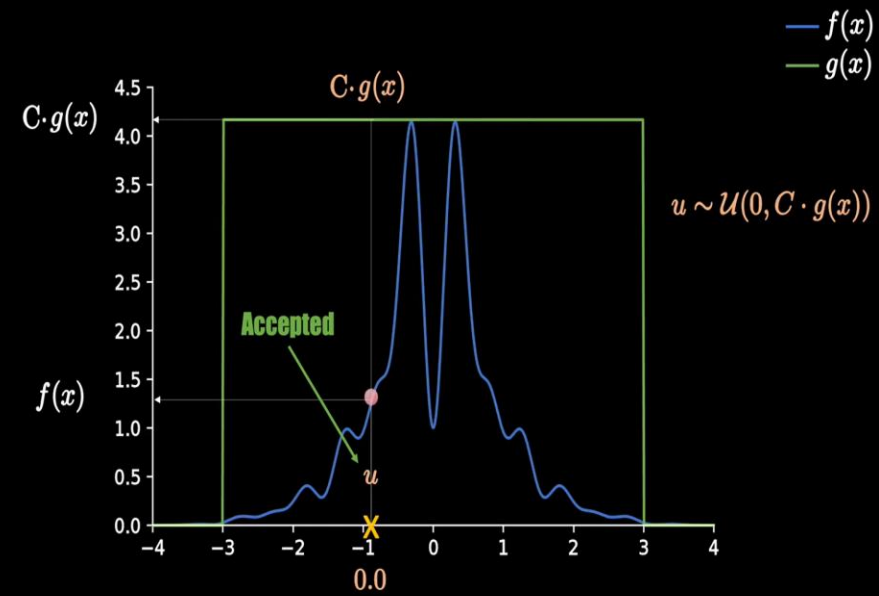
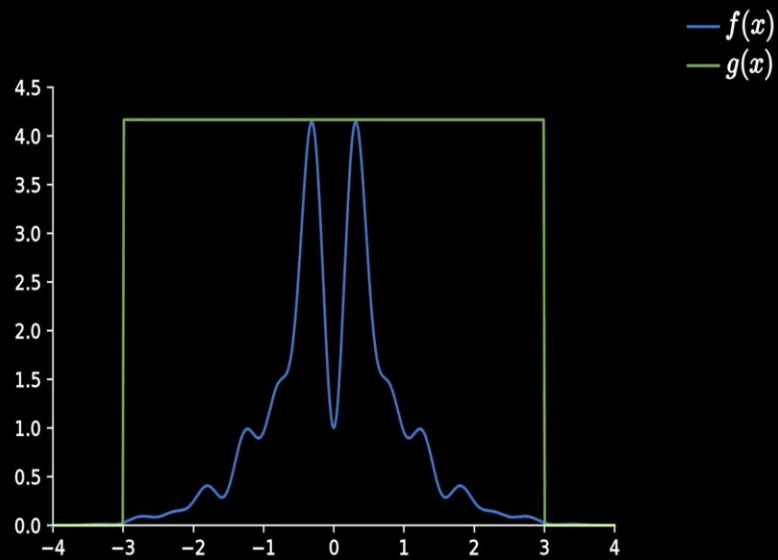
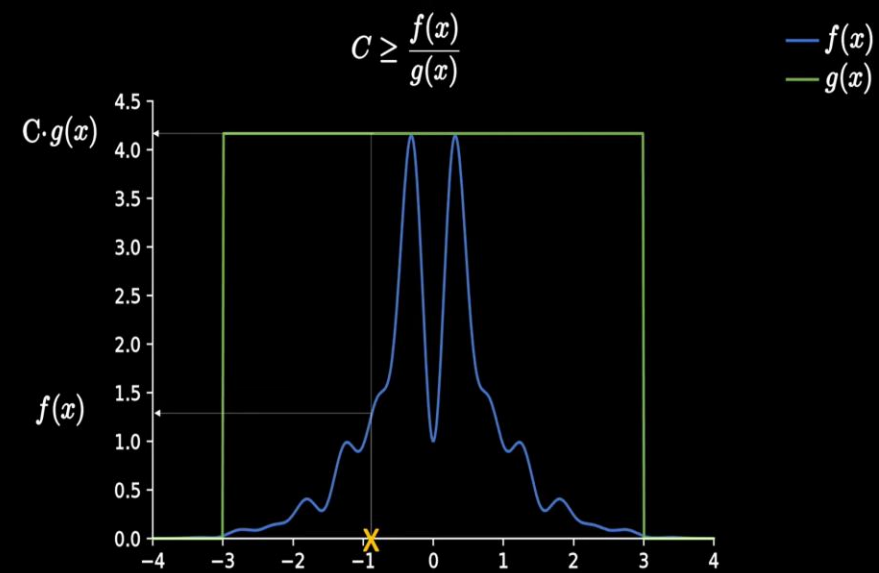


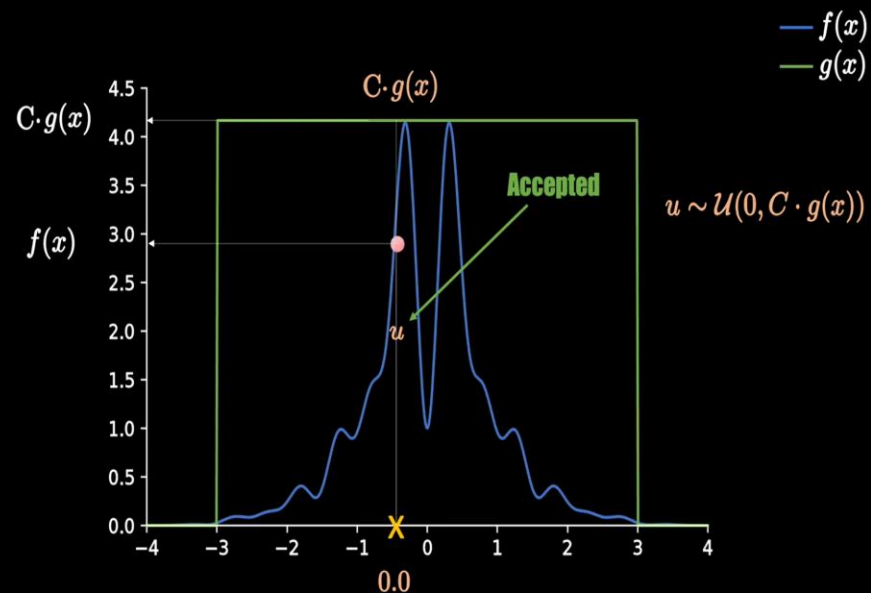
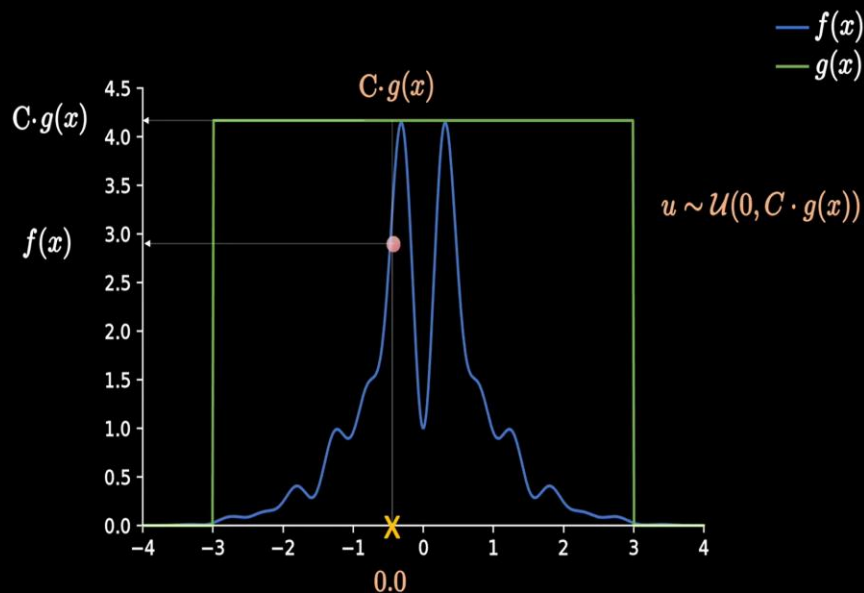
$$f(x) = e^{-x^2/2} \cdot (\sin^2(6+x) + 3 \cdot \cos^2(x) \cdot \sin^2(4x) + 1)$$



$$x \sim \mathcal{U}(-3, 3)$$

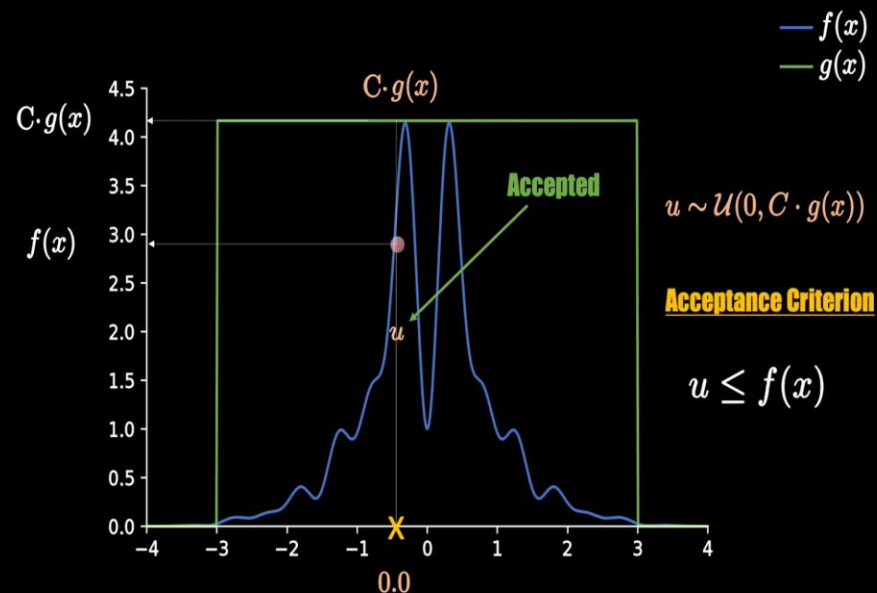
$$g(x) = \frac{1}{b-a}$$

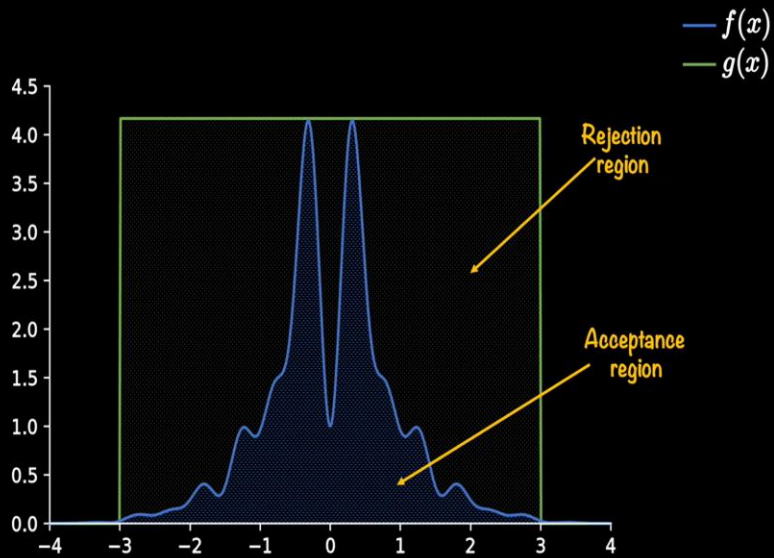




Rejection sampling –

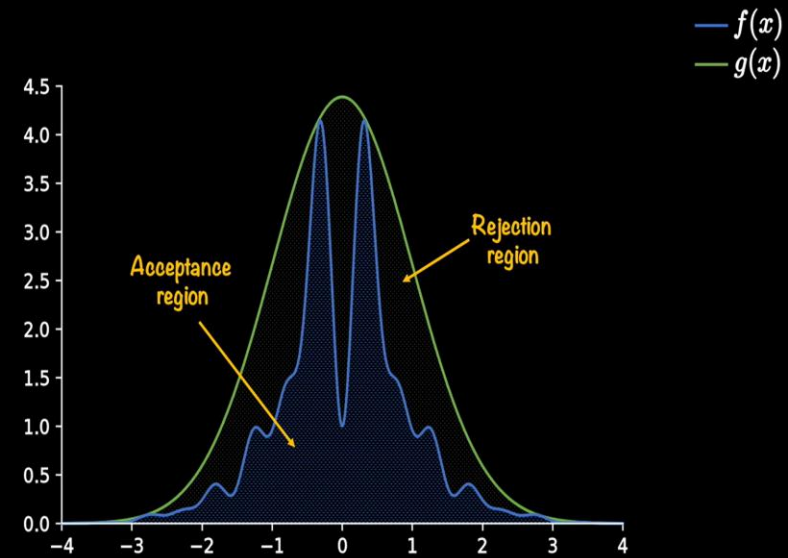
- Select a proposal distribution
- Scale it
- Use the acceptance criteria that involves randomness to make a decision





Run it many times ~1 mill

Acceptance rate = 23.5%



Acceptance rate = 53.63%

Why Use Rejection Sampling?

Efficiency: Rejection sampling is useful when directly sampling from the target distribution is difficult. It gives you an indirect way of generating samples that match the target distribution.

Flexibility: You can use any simple, easy-to-sample distribution as the proposal distribution, as long as you're able to "reject" the points that don't match the target.

However, rejection sampling can become inefficient if the proposal distribution is not well-aligned with the target distribution.

If too many samples get rejected, it can take a long time to collect enough good points.

Pros and cons of Monte Carlo method

Pros

Gives a tool estimate values – even if you are not an expert in that field
(as long as you have the rules of the problem)

“Easy”

Sometimes fast

Cons

Not always fast

I play a game with $P(\text{win}) = p$ for each round

Game ends if I lose 2 times in a row

$p, (1-p), (1-p)$

What is the expected number of rounds I play?

Let $e = E(R)$

```
rounds <- vector()
for (i in 1:1000000) {
  r <- 0
  nloss <- 0
```

$e = (1+e)$ $2 \times (2+e)$	p	True Answer	Monte Carlo Answer	Time (s)
$e = (2-p)$	0.5	6	5.996614	10.46
	0.75	20	19.979748	30.37
	0.9	110	109.976896	174.16

```
}
mean_rounds <- mean(rounds)
mean_rounds
```

If $p = 0.5$

true $e = 6$

estimated = 5.996614

Pros and cons of Monte Carlo method

Pros

Gives a tool estimate values – even if you are not an expert in that field

“Easy”

Sometimes fast

Cons

Not always fast

Not easily interpretable